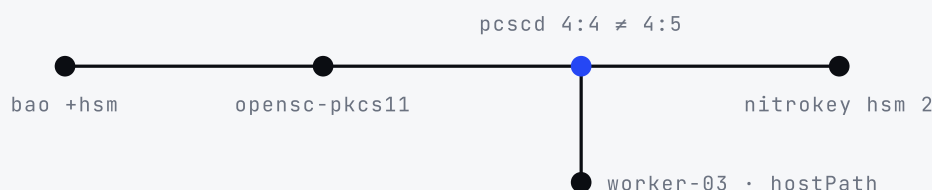


AUTO- UNSEAL OPENBAO

seal "pkcs11" mit einem USB-HSM am Worker: eigenes Image, Migration von Shamir – und sechs Fehler, von denen jeder den nächsten verdeckt hat.



Auto-Unseal mit einem Nitrokey HSM 2

seal “pkcs11” für den Cluster-OpenBao: der USB-Stick am Worker, ein eigenes Image, die Migration von Shamir – und sechs Fehler, von denen jeder den nächsten verdeckt hat

Thomas Zachmann

21. September 2026

Inhalt

Worum es geht

- Wie dieser Leitfaden entstanden ist
- Lizenz und Haftung
- Ein Wort zu den Werten in diesem Leitfaden
- Für wen
- Die Bausteine
- Die Architektur in einem Bild

Teil I – Seal, HSM, PKCS#11: die Begriffe

- Was ein Seal ist
- Was ein HSM dazu tut
- PKCS#11 – die Sprache
- DKEK – die Frage, die man vor dem ersten Schlüssel stellt

Teil II – Der Stick: Nitrokey HSM 2 am Worker

- USB-Durchreichung in Proxmox
- Pakete, pcscd, Polkit
- Den Token initialisieren – und zwar mit DKEK
- Was der Token kann – und was nicht
- Den Seal-Key erzeugen
- Die erste Probe aus einem Pod

Teil III – Das Image

- Warum ein eigenes Image
- Das Dockerfile
- Multi-Arch bauen und pushen
- Trivy
- Das Protokoll-Problem
- Das Label-Problem

Teil IV – Der Chart

- Die Values
- Das Secret mit der PIN
- Die Falle mit auditStorage
- Der Debug-Pod

Teil V – Die Migration

- Vorher: Snapshot
- Deploy und Neustart
- `unseal -migrate`
- Der Beweis
- Der Rückweg (nicht durchgeführt)

Teil VI – Was schiefging, und warum

Teil VII – Betrieb und Ausfallverhalten

Was jetzt ausfallen kann

Recovery Keys: was sie können und was nicht

Es sind die alten Keys – und warum man sie jetzt tauschen sollte

Der Schlüssel ist nicht gesichert

Snapshots bleiben Pflicht

Prüfen, dass es läuft

Teil VIII – Debug: Das Netzwerk-HSM antwortet nicht

Das Labor

Die Kette

Schritt 1 – Kommt man überhaupt hin?

Schritt 2 – Liegt die Bibliothek da, und lädt sie?

Schritt 3 – Sieht die Bibliothek einen Slot?

Schritt 4 – Kann das Token, was OpenBao braucht?

Schritt 5 – Login und Schlüssel

Schritt 6 – Eine Operation

Schritt 7 – Was das HSM selbst sagt

Schritt 8 – Die Stanza gegen das Gefundene halten

Schritt 9 – Mounts und Secrets

Schritt 10 – Jetzt die OpenBao-Logs

Schritt 11 – Von der Node ja, aus dem Pod nein

Schritt 12 – strace, und was es diesmal nicht zeigt

Nebenbei bewiesen: Recovery Keys öffnen nach manuellem Seal

Die Reihenfolge, als Tabelle

Was das Labor nicht beweist

Teil IX – Ein HSM für alle Pods: die HSM-VM (Entwurf)

Weg A – Transit-Seal über den VM-OpenBao (empfohlen)

Weg B – p11-kit remote: die PKCS#11-Bibliothek übers Netz

Weg C – USB/IP

Weg D – Nitrokey NetHSM

Empfehlung

Teil X – OpenBao 2.7: vom HSM-Build zum KMS-Plugin (Entwurf)

Die Stolpersteine, die man jetzt schon sieht

Anhang A – Alle Kommandos

Anhang B – Glossar

Über den Autor

Worum es geht

N° 1 endete mit einem Ritual: Nach jedem Pod-Neustart tippt jemand drei von fünf Unseal-Keys in `bao operator unseal`. Bis dahin ist OpenBao da, aber tot – 0/1 READY, kein Secret kommt heraus, jede Anwendung, die auf ihn wartet, wartet weiter. Ein Node-Update um drei Uhr nachts heißt: um drei Uhr nachts steht jemand auf.

Dieser Leitfaden schafft das Ritual ab. Der Root-Key, den bisher die Shamir-Keys rekonstruiert haben, liegt danach verschlüsselt mit einem Schlüssel, der einen **Nitrokey HSM 2** nie verlässt – ein USB-Stick, der am Cluster-Worker steckt. OpenBao startet, fragt das HSM, ist entsiegelt. Ohne Mensch, ohne Datei mit Keys, ohne Cloud-KMS.

Der Weg dahin war länger als geplant. Der vorbereitete Plan (N° 6 in der ersten Fassung) sah einen AES-Schlüssel auf dem HSM und den Stick an der VM aus N° 3 vor. Beides stimmte nicht: Der Nitrokey HSM 2 kann kein AES-GCM, und der Stick landete am Kubernetes-Worker. Das offizielle OpenBao-Image kann kein PKCS#11. Die PKCS#11-Bibliothek im eigenen Image sprach ein Protokoll, das der Daemon auf dem Host ablehnt. Der Token hieß anders, als das Test-Werkzeug behauptete. Und ein leeres Secret sah aus wie ein volles. Sechs Fehler, jeder einzeln trivial, zusammen ein Nachmittag – und der eigentliche Inhalt dieses Leitfadens.

Es ist der sechste Teil einer Reihe. Er setzt den Cluster-OpenBao aus N° 1 voraus und den Snapshot aus N° 7, ohne den man eine Seal-Migration nicht anfangen sollte. Teil VIII wechselt das Gerät: Ein Netzwerk-HSM – Nitrokeys NetHSM als Test-Container – wird im Cluster aufgesetzt und Schicht für Schicht kaputtgemacht, als Debug-Verfahren mit echten Ausgaben. Zwei Teile am Ende sind Entwürfe und als solche markiert: ein HSM, das nicht an einem Worker hängt, sondern allen Pods dient – und der Weg, den OpenBao 2.7 vorgibt, wenn es die HSM-Distribution einstellt.

Wie dieser Leitfaden entstanden ist

Die anderen Nummern dieser Reihe sind ohne KI-Assistenz geschrieben. Diese nicht: Der Umbau am Cluster lief in einer Pair-Programming-Sitzung mit Claude Code, das Recherche, Debugging-Pods und den ersten Entwurf des Texts beigesteuert hat. Jeder Befehl wurde am Cluster ausgeführt, jede Ausgabe ist echt, jede Entscheidung habe ich getroffen. Der Maßstab bleibt derselbe: Wer den Leitfaden durchgearbeitet hat, kann auf einem leeren Blatt die Kette vom bao-Prozess bis zum Chip im Stick aufzeichnen und sagen, an welchen vier Stellen sie reißen kann.

Lizenz und Haftung

Dieser Leitfaden steht unter CC BY 4.0: Er darf kopiert, weitergegeben und bearbeitet werden, auch kommerziell, solange der Autor genannt wird. Er wird ohne Gewähr bereitgestellt. Alles darin wurde in einer Entwicklungsumgebung durchgeführt; wer es in einer anderen Umgebung nachvollzieht, tut das auf eigene Verantwortung. Ein HSM in Produktion zu betreiben braucht mehr, als hier steht – Teil VII sagt, was.

Ein Wort zu den Werten in diesem Leitfaden

Die User-PIN des HSM, die SO-PIN, die Unseal-Keys und der Root-Token erscheinen nirgends – auch nicht als Entwicklungswerte. Die PIN ging als Datei in ein Kubernetes-Secret und die Datei danach in `rm -P`. Interne Adressen, der externe Hostname und die Seriennummer des Sticks sind durch Platzhalter ersetzt (`<worker-03-ip>`, `openbao.example.internal`, `DENK0000000`). Das Image `softxpert/openbao-pkcs11` ist öffentlich und echt.

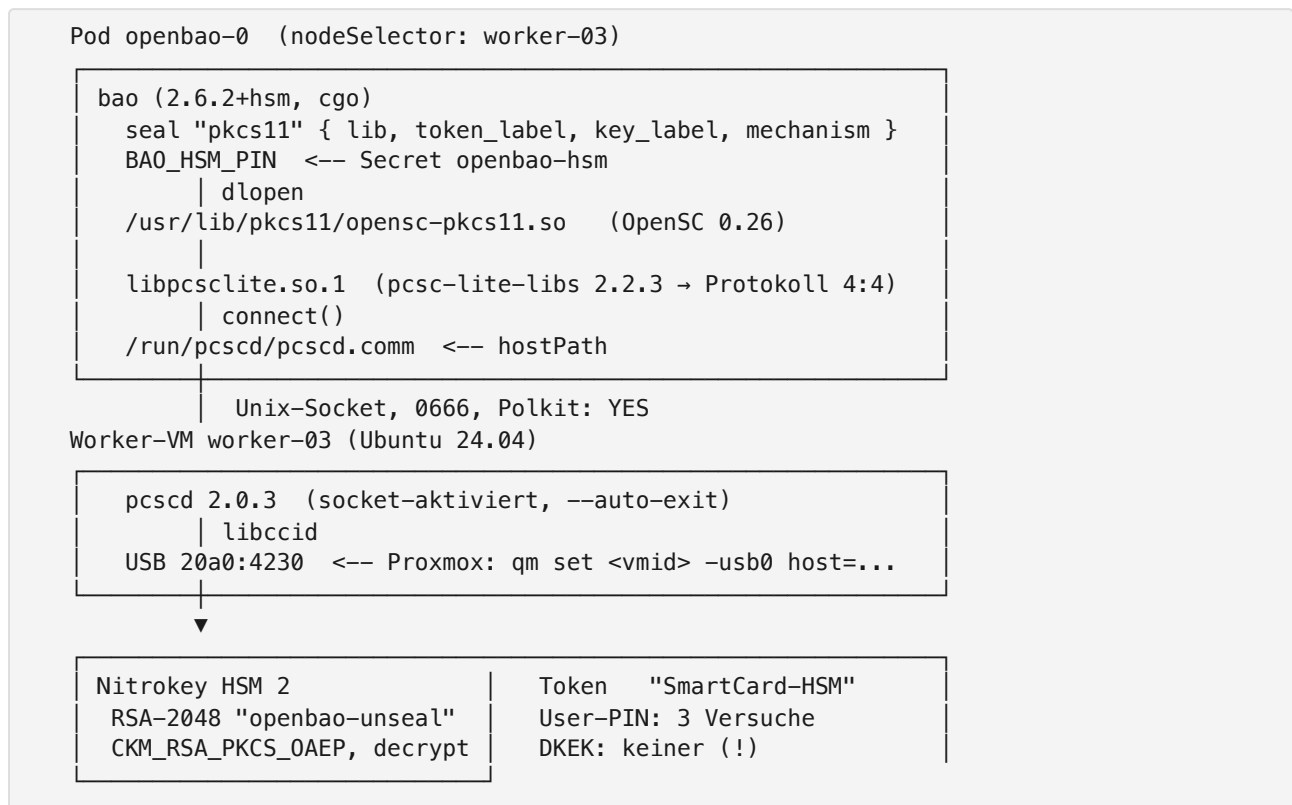
Für wen

Für Leserinnen und Leser, die N° 1 kennen und noch nie ein HSM in der Hand hatten. Teil I erklärt, was ein Seal ist, was ein HSM dazu tut und was die Begriffe PKCS#11, Token, Slot, SO-PIN und DKEK bedeuten – wer das weiß, überspringt ihn. OpenBao-Grundlagen liefert mein Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice).

Die Bausteine

Baustein	Version	Rolle
OpenBao im Cluster (N° 1)	2.6.2, Chart 0.29.4, standalone, Raft	das, was entsiegelt werden soll
Nitrokey HSM 2	Firmware 4.1 (SmartCard-HSM)	hält den Seal-Key <code>openbao-unseal</code> , RSA 2048
Proxmox	–	reicht den USB-Stick per <code>qm set -usb0</code> an die Worker-VM durch
Worker-VM <code>worker-03</code>	Ubuntu 24.04, <code>pcscd</code> 2.0.3, <code>libccid</code> , OpenSC 0.25	spricht mit dem Stick; <code>pcscd</code> hört auf <code>/run/pcscd/pcscd.comm</code>
Image <code>softxpert/openbao-pkcs11:2.6.2</code>	<code>openbao/openbao-hsm:2.6.2</code> + OpenSC 0.26 + <code>pcsc-lite-libs 2.2.3</code>	OpenBao mit PKCS#11-Support und der Client-Bibliothek
Helm-Values	<code>server.image</code> , <code>nodeSelector</code> , <code>hostPath /run/pcscd</code> , <code>seal "pkcs11"</code> , <code>extraSecretEnvironmentVars</code>	bringt alles zusammen
Secret <code>openbao-hsm</code>	<code>BA0_HSM_PIN</code> , 8 Bytes	die User-PIN, nur im Cluster

Die Architektur in einem Bild



Drei Dinge, die man aus dem Bild mitnehmen sollte:

1. **Die Kette hat vier Schnittstellen, und jede hat einen eigenen Fehlermodus.** bao ↔ Bibliothek (Build ohne PKCS#11), Bibliothek ↔ Daemon (Protokollversion), Daemon ↔ Reader (Polkit, CCID), Reader ↔ Chip (PIN, Label, Key). Teil VI hat für jede einen Eintrag.
2. **Der Pod ist an den Worker gebunden.** Ein USB-Stick folgt keinem Pod. Stirbt worker-03, gibt es keinen Unseal – nicht mit Recovery Keys, mit nichts. Das ist ein bewusster Single Point of Failure, kein Versehen, und Teil IX zeigt, wie man ihn loswird.
3. **Der Schlüssel ist nicht gesichert.** Der Token wurde ohne DKEK initialisiert. Der Seal-Key existiert genau einmal, im Chip. Teil VII sagt, was das bedeutet und was vor dem Produktiveinsatz zu tun ist.

Teil I – Seal, HSM, PKCS#11: die Begriffe

Was ein Seal ist

OpenBao verschlüsselt alles, was es speichert, mit einem **Root-Key** (ältere Texte sagen Master-Key). Der Root-Key liegt selbst im Storage – verschlüsselt. Womit, das entscheidet der **Seal**:

- **Shamir** (N° 1): Der Root-Key wird nach Shamirs Secret Sharing in fünf Teile zerlegt, drei genügen zur Rekonstruktion. Die Teile sind die Unseal-Keys. Nach jedem Start müssen drei davon eingegeben werden; erst dann kann OpenBao den Root-Key bilden und die Barrier öffnen.
- **Auto-Seal**: Der Root-Key ist mit einem Schlüssel verschlüsselt, der außerhalb von OpenBao liegt – in einem Cloud-KMS, in einem zweiten OpenBao (Transit) oder in einem HSM. OpenBao fragt beim Start dieses System, bekommt den Root-Key entschlüsselt zurück und öffnet sich selbst.

Beim Auto-Seal bleiben die fünf Keys erhalten, heißen aber **Recovery Keys** und tun etwas anderes: Sie autorisieren einen Root-Token-Reset und den Rückweg zu Shamir. Den Root-Key können sie **nicht** rekonstruieren. Wer sein HSM verliert, hat mit Recovery Keys einen Tresor ohne Schlüssel.

Was ein HSM dazu tut

Ein **Hardware Security Module** ist ein Gerät, das kryptografische Schlüssel erzeugt, speichert und benutzt, ohne sie je herauszugeben. Man schickt ihm Daten, es antwortet mit dem Ergebnis – signiert, entschlüsselt, gewrappt. Der private Schlüssel verlässt den Chip nicht; eine Kopie anzufertigen ist nicht vorgesehen, das Auslesen soll auch physisch scheitern.

Der Nitrokey HSM 2 ist die kleine Form davon: ein USB-Stick mit einer SmartCard-HSM-Karte von CardContact. Er tut, was ein Rack-HSM tut, langsam und für einen Schlüsselsatz. Für einen Seal-Key reicht das – OpenBao braucht das HSM nur beim Start und alle zehn Minuten für einen Health-Check.

PKCS#11 – die Sprache

PKCS#11 (auch „Cryptoki“) ist die Standard-API, über die Programme mit HSMs und Smartcards sprechen. Sie ist eine C-Bibliothek – eine .so-Datei –, die ein Programm per dlopen lädt. Die Begriffe darin:

- **Slot** – ein Steckplatz, physisch oder logisch. Hier: der USB-Reader.
- **Token** – die Karte im Slot. Sie hat ein **Label** (einen Namen), eine Seriennummer und Flags („login required“, „PIN initialized“).
- **Objekte** – was auf dem Token liegt: Schlüssel, Zertifikate, Daten. Jedes Objekt hat ein Label und eine ID. Private Schlüssel sind nur nach Login sichtbar.
- **Mechanismen** – was das Token rechnen kann: CKM_RSA_PKCS_OAEP, CKM_AES_GCM, ... Jedes Token kann eine andere Auswahl. Was es nicht kann, kann OpenBao nicht mit ihm.
- **User-PIN** – der Login für Schlüsseloperationen. Drei Fehlversuche, dann ist sie gesperrt.

- **SO-PIN** (Security Officer) – der Verwaltungs-Login: Token initialisieren, User-PIN zurücksetzen, Token löschen. 15 Versuche. OpenBao bekommt sie nie zu sehen.

Für den Nitrokey HSM 2 liefert **OpenSC** die PKCS#11-Bibliothek (`opensc-pkcs11.so`) und die Werkzeuge (`pkcs11-tool`, `sc-hsm-tool`, `opensc-tool`). OpenSC spricht nicht selbst mit USB, sondern mit dem **PC/SC-Daemon** `pcscd` über einen Unix-Socket; `pcscd` spricht per **CCID**-Treiber mit dem Reader. Vier Schichten, alle müssen passen.

DKEK – die Frage, die man vor dem ersten Schlüssel stellt

Ein HSM gibt Schlüssel nicht heraus. Was, wenn es kaputtgeht? Der SmartCard-HSM hat dafür den **Device Key Encryption Key**: Wird der Token mit einem oder mehreren DKEK-Shares initialisiert, kann man Schlüssel später mit dem DKEK verschlüsselt exportieren (`sc-hsm-tool --wrap-key`) und auf einen zweiten, mit denselben Shares initialisierten Stick importieren. Ohne DKEK: kein Export, kein Ersatzstick, keine Sicherung. Die Entscheidung fällt beim `--initialize` und lässt sich danach nur durch Neu-Initialisieren ändern – die alle Schlüssel löscht.

Der Token in diesem Leitfaden wurde **ohne** DKEK initialisiert. Was das bedeutet, steht in Teil VII. Wer es nachbaut, sollte es anders machen, und Teil II zeigt wie.

Teil II – Der Stick: Nitrokey HSM 2 am Worker

USB-Durchreichung in Proxmox

Die Worker sind VMs. Der Stick steckt im Proxmox-Host und muss an die VM gereicht werden – einmal, auf dem Host, bevor auf der VM etwas passiert:

```
# auf dem Proxmox-Host
lsusb | grep -i nitrokey
#   Bus 001 Device 005: ID 20a0:4230 Clay Logic Nitrokey HSM

# an die VM binden – nach Vendor:Product, nicht nach Bus/Port: überlebt
# ein Umstecken in einen anderen Port
qm set <vmid> –usb0 host=20a0:4230

# Kaltstart ist der zuverlässige Weg; Hot-Plug geht auf neuem QEMU, ist
# für ein HSM aber die Unsicherheit nicht wert
qm shutdown <vmid> && qm start <vmid>
```

In der VM:

```
$ lsusb | grep -i nitrokey
Bus 002 Device 002: ID 20a0:4230 Clay Logic Nitrokey HSM
```

Pakete, pcsd, Polkit

```
sudo apt install -y pcsd libccid pcsc-tools opensc
sudo systemctl enable --now pcsd
```

Ubuntu 24.04 liefert pcsd 2.0.3, libccid 1.5.5, OpenSC 0.25. pcsd ist **socket-aktiviert**: pcsd.socket hört auf /run/pcsd/pcsd.comm, der Dienst startet beim ersten Client und beendet sich nach Leerlauf wieder (--auto-exit). systemctl is-active pcsd zeigt deshalb oft inactive, während pcsd.socket active ist – das ist kein Fehler.

```
$ pcsc_scan
Reader 0: Nitrokey Nitrokey HSM (DENK00000000) 00 00
Card state: Card inserted,
ATR: 3B DE 18 FF 81 91 FE 1F C3 80 31 81 54 48 53 4D 31 73 80 21 40 81 07 FA
```

Wer nicht Root ist, sieht zunächst nichts: pcsd fragt **Polkit**, ob der anfragende Prozess den Reader benutzen darf, und die Voreinstellung sagt Nein für alles, was nicht in einer lokalen Sitzung sitzt. Ein Container-Prozess sitzt in keiner. Die Regel, die das öffnet:

```
// /etc/polkit-1/rules.d/40-allow-pcsd.rules
polkit.addRule(function(action, subject) {
    if (
        action.id == "org.debian.pcsd-lite.access_pcsd" ||
        action.id == "org.debian.pcsd-lite.access_card"
    ) {
        return polkit.Result.YES;
    }
});
```

```
sudo systemctl restart polkit pcsd
```

Das ist bewusst grob: Jeder Prozess auf dem Host darf zum Reader. Für einen Worker, auf dem ohnehin nur Pods laufen, vertretbar; auf einer Mehrbenutzer-Maschine würde man `subject.user` prüfen. Der Socket selbst ist `0666 (srw-rw-rw-)`, die eigentliche Zugriffskontrolle ist Polkit.

Den Token initialisieren – und zwar mit DKEK

Ein fabrikneuer Stick hat eine Transport-SO-PIN (3537363231383830) und muss initialisiert werden. Dabei fallen drei Entscheidungen: SO-PIN, User-PIN, DKEK-Shares. **Das hier ist der Schritt, der in diesem Leitfaden anders gelaufen ist als er sollte** – der Token wurde ohne DKEK-Shares initialisiert. So sähe es richtig aus:

```
# SO-PIN: 16 Hex-Zeichen. User-PIN: 6–15 Zeichen. Beide werden abgefragt,
# nicht als Argument übergeben (Shell-History!).
sc-hsm-tool --initialize --dkek-shares 1 --label openbao

# den DKEK-Share erzeugen und importieren – die Share-Datei ist das Backup
# des HSM. Sie ist passwortgeschützt; Datei und Passwort getrennt aufheben.
sc-hsm-tool --create-dkek-share dkek-share-1.pbe
sc-hsm-tool --import-dkek-share dkek-share-1.pbe

sc-hsm-tool                                # muss "DKEK shares : 1" zeigen
```

Mit DKEK lässt sich jeder später erzeugte Schlüssel als verschlüsselte Datei exportieren (`--wrap-key`) und auf einen Ersatzstick importieren, der mit derselben Share-Datei initialisiert wurde. Ohne DKEK gibt es keinen Ersatzstick.

Der Zustand hier:

```
$ sc-hsm-tool
Using reader with a card: Nitrokey Nitrokey HSM (DENK00000000 ) 00 00
Version                : 4.1
Config options         :
    User PIN reset with SO-PIN enabled
SO-PIN tries left      : 15
User PIN tries left    : 3
```

Keine Zeile DKEK shares. Was auf diesem Token liegt, liegt nur dort.

Was der Token kann – und was nicht

Bevor man einen Schlüssel erzeugt, fragt man, welche Mechanismen das Token beherrscht. OpenBao braucht für den Seal ein AEAD-Verfahren: CKM_AES_GCM oder CKM_RSA_PKCS_OAEP.

```
$ pkcs11-tool -M | grep -E 'AES|OAEP'  
RSA-PKCS-OAEP, keySize={1024,4096}, hw, decrypt
```

Kein AES-GCM. Der vorbereitete Plan – AES-256, mechanism = "0x1087" – wäre am Token gescheitert. RSA-OAEP ist da, mit decrypt in Hardware. So funktioniert der Seal dann: OpenBao erzeugt einen zufälligen AES-Schlüssel, verschlüsselt damit den Root-Key, wrappt den AES-Schlüssel mit dem öffentlichen RSA-Schlüssel (in Software) und speichert beides. Beim Unseal schickt es den gewrappten AES-Schlüssel ans HSM, das ihn mit dem privaten RSA-Schlüssel entschlüsselt. Der private Schlüssel bleibt im Chip.

Den Seal-Key erzeugen

Ein RSA-2048-Paar, Label openbao-unseal, ID 10, Nutzung decrypt:

```
pkcs11-tool --login --keypairgen --key-type rsa:2048 \  
--label openbao-unseal --id 10 --usage-decrypt
```

Die User-PIN wird abgefragt. Danach – ohne Login sieht man nur den öffentlichen Teil, das ist so gewollt:

```
$ pkcs11-tool -0  
Using slot 0 with a present token (0x0)  
Public Key Object; RSA 2048 bits  
label:      openbao-unseal  
ID:         10  
Usage:      encrypt, verify, wrap
```

(Der Schlüssel dieses Leitfadens wurde vor der protokollierten Sitzung erzeugt; das Kommando oben ist das Standardverfahren, das zu genau diesem Objekt führt.)

Die erste Probe aus einem Pod

Bevor OpenBao ins Spiel kommt: Sieht ein Container den Stick? Ein Wegwerf-Pod auf worker-03 mit dem pcsd-socket als hostPath:

```

apiVersion: v1
kind: Pod
metadata: {name: pkcs11-test, namespace: openbao}
spec:
  nodeSelector: {kubernetes.io/hostname: worker-03}
  securityContext: {runAsNonRoot: true, runAsUser: 1000, runAsGroup: 1000}
  containers:
  - name: test
    image: debian:bookworm-slim          # später: eigenes Image mit opensc
    command: ["sleep", "infinity"]
    volumeMounts: [{name: pcscd, mountPath: /run/pcscd}]
  volumes:
  - name: pcscd
    hostPath: {path: /run/pcscd, type: Directory}

```

```

$ kubectl exec -n openbao pkcs11-test -- pkcs11-tool -L
Available slots:
Slot 0 (0x0): Nitrokey Nitrokey HSM (DENK00000000 ) 00 00
  token label      : SmartCard-HSM (UserPIN)
  token manufacturer : www.CardContact.de
  token model      : PKCS#15 emulated
  token flags      : login required, rng, token initialized, PIN initialized

```

Funktioniert – als UID 1000, aus Debian bookworm, mit OpenSC 0.23. Dieses Ergebnis hat später zwei falsche Sicherheiten erzeugt: dass die Kette Pod → Socket → pcscd grundsätzlich geht (stimmt), und dass der Token SmartCard-HSM (UserPIN) heißt (stimmt nur für diese OpenSC-Version).

Teil III – Das Image

Warum ein eigenes Image

Das offizielle openbao/openbao enthält kein OpenSC und – wichtiger – ein bao-Binary, das ohne PKCS#11-Support kompiliert ist. Der Seal braucht CGO und den Build-Tag hsm. OpenBao liefert dafür eine zweite Distribution: openbao/openbao-hsm (Alpine) und openbao/openbao-hsm-ubi (RHEL UBI, glibc). Beide auf Docker Hub und Quay, amd64 und arm64.

Der erste Versuch nahm die falsche Basis. Das Dockerfile war formal korrekt, das Image baute, der Pod startete – und starb:

```
Error configuring seal "pkcs11": this build of OpenBao has PKCS#11 disabled
```

Die Bibliotheken waren da. Das Programm, das sie laden sollte, konnte es nicht. Die Meldung ist eindeutig, wenn man weiß, dass es zwei Builds gibt.

Das Dockerfile

```
FROM openbao/openbao-hsm:2.6.2

USER root

# pcsc-lite-libs bewusst auf 2.2.x aus Alpine 3.21 gepinnt: pcsc-lite >= 2.3
# spricht Client-Protokoll 4:5, pcscd 2.0.3 auf dem Ubuntu-24.04-Worker
# akzeptiert nur 4:4 und lehnt neuere Clients ab ("Communication protocol
# mismatch"). Prüfen, sobald der pcscd des Workers oder das Basis-Image wechselt.
RUN apk add --no-cache \
    --repository https://dl-cdn.alpinelinux.org/alpine/v3.21/main \
    opensc \
    'pcsc-lite-libs=2.2.3-r1'

USER openbao
```

Drei Zeilen tun etwas, und jede hat eine Geschichte:

- FROM openbao/openbao-hsm:2.6.2 – der Build mit PKCS#11. bao version sagt OpenBao v2.6.2+hsm ... (cgo). Alpine 3.24, User openbao mit UID 100 – dieselbe UID wie im Standard-Image, nichts im Chart muss sich ändern.
- opensc – die PKCS#11-Bibliothek /usr/lib/pkcs11/opensc-pkcs11.so und die Werkzeuge. Alpine zieht pcsc-lite-libs nicht automatisch nach; ohne das Paket meldet OpenSC „Unable to load external module“.
- pcsc-lite-libs=2.2.3-r1 aus dem **3.21**-Repository statt 2.4.0 aus 3.24 – der zweite Fehler dieses Leitfadens, Erklärung folgt.

Multi-Arch bauen und pushen

Der Cluster ist amd64, der Mac arm64. `docker build` liefert nur die Host-Architektur; für beides braucht es BuildKit:

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t softxpert/openbao-pkcs11:2.6.2 --push .
```

`--push` ist hier kein Komfort: Ein Multi-Arch-Image ist eine Manifest-Liste, die auf ein Image pro Plattform zeigt. Der klassische lokale Image-Store kann so etwas nicht halten; BuildKit exportiert direkt in die Registry. Der amd64-Teil entsteht auf dem Mac per QEMU-Emulation – für `apk add` unkritisch.

```
$ docker buildx imagetools inspect softxpert/openbao-pkcs11:2.6.2
Digest:      sha256:265babb4f237b03cbcd94abaaf84a9cb5f75d74acecf88a867685f792564e2c5
Platform:    linux/amd64
Platform:    linux/arm64
```

Der Digest ist die Referenz, die in die Values gehört – nicht der Tag. Der Tag 2.6.2 wurde in dieser Sitzung dreimal überschrieben (falsche Basis, richtige Basis, gepinnte Bibliothek). Mit `pullPolicy: IfNotPresent` hätte der Worker jedes Mal die gecachte alte Kopie behalten.

Trivy

```
$ trivy image --platform linux/amd64 --severity HIGH,CRITICAL softxpert/openbao-pkcs11:2.6.2
softxpert/openbao-pkcs11:2.6.2 (alpine 3.24.1)  2 HIGH  (libcrypto3/libssl3, OpenSSL QUI
C-DoS, fixed in 3.5.8-r0)
usr/bin/bao (gobinary)                          9 HIGH
```

Alle elf Funde stammen aus dem Basis-Image; `opensc` und `pcsc-lite-libs` haben keinen einzigen hinzugefügt. Fünf der neun im Go-Binary sind False Positives: Trivy meldet OpenBao-CVEs, die laut eigener Tabelle in 2.0.3 bis 2.5.4 gefixt sind – das Binary trägt aber eine Go-Pseudo-Version (`v0.0.0-20260818...`) statt `v2.6.2`, und Trivy kann sie nicht vergleichen. Die zwei OpenSSL-Funde ließen sich mit `apk upgrade --no-cache` beheben.

Das Protokoll-Problem

Das Image mit der richtigen Basis startete, fand den PIN – und dann:

```
Error configuring seal "pkcs11": failed to find token with label: SmartCard-HSM (UserPIN)
```

Der Container lebte zu kurz für ein `kubectl exec`. Also ein Debug-Pod mit **demselben Image, derselben UID, denselben Mounts, demselben Node**, der nur `sleep` ausführt:

```
$ kubectl exec -n openbao pkcs11-debug -- opensc-tool -l
No smart card readers found.
```

Derselbe Socket, an dem der Debian-Test-Pod Minuten vorher den Stick sah. Die Antwort stand im Journal des Hosts:

```
$ sudo journalctl -u pcscd --since -15min
pcscd[530642]: winscard_svc.c:402:ContextThread() Communication protocol mismatch!
pcscd[530642]: winscard_svc.c:404:ContextThread() Client protocol is 4:5
pcscd[530642]: winscard_svc.c:406:ContextThread() Server protocol is 4:4
```

pcsc-lite hat mit Version 2.3.0 die Minor-Version seines Client-Server- Protokolls angehoben, und pcscd lehnt Clients mit anderer Minor-Version ab. Empirisch, mit Wegwerf-Pods aus verschiedenen Alpine-Versionen gegen denselben Host:

Alpine	pcsc-lite-libs	Protokoll	gegen pcscd 2.0.3
3.21	2.2.3	4:4	✅ Token sichtbar
3.22	2.3.3	4:5	❌ „No smart card readers“
3.24	2.4.0	4:5	❌
Debian bookworm	1.9.9	4:4	✅ (der Test-Pod)

Ubuntu 24.04 bleibt bei pcscd 2.0.3 – der Host ist nicht ohne Fremdpakete zu heben. Die Client-Seite ist es: eine Zeile im Dockerfile. Das ist ein Cross-Release-Paket und gehört mit Kommentar versehen, damit beim nächsten Basis-Image-Wechsel jemand nachschaut, ob es noch nötig ist.

Das Label-Problem

Mit gepinnter Bibliothek, aus dem Debug-Pod:

```
$ kubectl exec -n openbao pkcs11-debug -- pkcs11-tool -L
Slot 0 (0x0): Nitrokey Nitrokey HSM (DENK00000000) 00 00
token label      : SmartCard-HSM
```

Nicht SmartCard-HSM (UserPIN). Das Label ist kein Attribut des Chips, sondern wird von OpenSCs PKCS#15-Emulation erzeugt – und OpenSC 0.26 erzeugt ein anderes als 0.23 und 0.25. token_label in der Seal-Stanza muss das sein, was **die OpenSC-Version im Image** meldet. Nach jedem Rebuild prüfen. Die Alternative slot = "0" ist bei einem einzigen Reader stabil, aber weniger lesbar.

Teil IV – Der Chart

Die Values

Die Änderungen an helm/values.yaml aus N° 1, alles unter server::

```
server:
  # Chart-Default-Registry ist quay.io – ohne "registry" würde
  # quay.io/softxpert/openbao-pkcs11 gezogen: ImagePullBackOff.
  image:
    registry: docker.io
    repository: softxpert/openbao-pkcs11
    tag: "2.6.2@sha256:265babb4f237b03cbcd94abaaf84a9cb5f75d74acecf88a867685f792564e2c5"
    pullPolicy: IfNotPresent

  # Der Stick hängt an worker-03. Bewusster SPOF (Teil IX).
  nodeSelector:
    kubernetes.io/hostname: worker-03

  # Das Verzeichnis mounten, nicht die Socket-Datei: pcsd legt den Socket
  # bei jedem Start neu an; ein Datei-Mount zeigte auf den toten Inode.
  # Kein readOnly – connect() auf einen Unix-Socket braucht Schreibrecht.
  volumes:
    - name: pcsd
      hostPath:
        path: /run/pcsd
        type: Directory
  volumeMounts:
    - name: pcsd
      mountPath: /run/pcsd

  standalone:
    enabled: true
    config: |
      ui = true
      listener "tcp" { ... }           # wie N° 1
      storage "raft" { ... }           # wie N° 1

      # token_label ist, was OpenSC *in diesem Image* meldet (pkcs11-tool -L):
      # 0.26 sagt "SmartCard-HSM", ältere "SmartCard-HSM (UserPIN)".
      seal "pkcs11" {
        lib           = "/usr/lib/pkcs11/opensc-pkcs11.so"
        token_label   = "SmartCard-HSM"
        key_label     = "openbao-unseal"
        mechanism     = "CKM_RSA_PKCS_OAEP"
      }

  # Die PIN kommt aus dem Secret, nie aus der Datei
  extraSecretEnvironmentVars:
    - envName: BAO_HSM_PIN
      secretName: openbao-hsm
      secretKey: BAO_HSM_PIN
```

Vier Punkte dazu:

- **tag mit Digest.** Der Chart rendert `registry/repository:tag`; ein tag der Form `2.6.2@sha256:...` ergibt eine gültige Referenz, bei der der Digest gewinnt. Kein neuer Chart-Parameter nötig.
- **pin steht nicht in der Stanza.** Jeder Seal-Parameter hat eine Umgebungsvariable (`BAO_HSM_PIN`, `BAO_HSM_LIB`, ...), und die Variable hat Vorrang. Die ConfigMap mit der Stanza liegt lesbar im Cluster; das Secret nicht.
- **mechanism ist explizit**, obwohl OpenBao es aus dem Schlüsseltyp ableiten könnte. Im Repo soll stehen, was das HSM tut.
- **updateStrategyType: OnDelete** ist Chart-Default. Ein `helm upgrade` ändert das `StatefulSet`-Template, ersetzt aber keinen laufenden Pod. Jede Änderung hier wird erst mit `kubectl delete pod openbao-0` wirksam.

Das Secret mit der PIN

```
# in einem eigenen Terminal, zsh:
read -rs "PIN?HSM User-PIN: "; printf '%s' "$PIN" > /tmp/hsm-pin; unset PIN
wc -c < /tmp/hsm-pin                # 6-15, sonst stimmt etwas nicht

kubectl -n openbao create secret generic openbao-hsm \
  --from-file=BAO_HSM_PIN=/tmp/hsm-pin
rm -P /tmp/hsm-pin                  # macOS; Linux: shred -u

# und PRÜFEN – kubectl validiert Werte nicht:
kubectl -n openbao get secret openbao-hsm -o json \
  | python3 -c "import sys,json,base64; d=json.load(sys.stdin)['data']; print({k: len(base64.b64decode(v)) for k,v in d.items()})"
{'BAO_HSM_PIN': 8}
```

Die letzte Zeile ist kein Zierrat. Der erste Versuch dieses Leitfadens erzeugte ein Secret mit **0 Bytes**: Die Anleitung sagte `read -rs -p "Prompt" PIN` – Bash-Syntax. In zsh bedeutet `read -p` „vom Koprozess lesen“, `$PIN` blieb leer, `--from-literal=BAO_HSM_PIN=""` legte ein leeres Secret an, und OpenBao sagte `pin is required`. Das Secret existierte, hatte den richtigen Key, sah in `kubectl get secret` aus wie jedes andere.

Die Falle mit auditStorage

Nicht Teil des Seals, aber Teil dieses Umbaus: Die Values bekamen ein `auditStorage` (eigenes Volume für das Audit-Device). Das erzeugt ein zweites `volumeClaimTemplate` im `StatefulSet` – und Kubernetes verbietet das bei einem bestehenden `StatefulSet`:

```
updates to statefulset spec for fields other than 'replicas', 'ordinals',
'template', 'updateStrategy', 'persistentVolumeClaimRetentionPolicy' and
'minReadySeconds' are forbidden
```

`helm --dry-run=server` sieht das **nicht** – es rendert mit Server-Lookups, schickt die Manifeste aber nicht mit `dryRun` an den API-Server. Der Weg: `kubectl delete sts openbao --cascade=orphan` (der Pod läuft weiter), `helm upgrade` legt das `StatefulSet` neu an und adoptiert den Pod über die Labels, dann `delete pod`. Die PVC bleibt dank `persistentVolumeClaimRetentionPolicy: Retain`.

Der Debug-Pod

Das wichtigste Werkzeug dieses Leitfadens, für den Anhang:

```
apiVersion: v1
kind: Pod
metadata: {name: pkcs11-debug, namespace: openbao}
spec:
  nodeSelector: {kubernetes.io/hostname: worker-03}
  restartPolicy: Never
  securityContext: {runAsNonRoot: true, runAsUser: 100, runAsGroup: 1000}
  containers:
  - name: debug
    image: docker.io/softxpert/openbao-pkcs11:2.6.2@sha256:265babb4...
    command: ["sleep", "1800"]
    volumeMounts: [{name: pcscd, mountPath: /run/pcscd}]
  volumes:
  - name: pcscd
    hostPath: {path: /run/pcscd, type: Directory}
```

Gleiches Image, gleiche UID 100, gleicher Mount, gleicher Node – nur `sleep` statt `bao`. Darin `opensc-tool -l, pkcs11-tool -L, -0, -M`. Was hier geht, geht auch in OpenBao; was hier scheitert, scheitert dort mit einer schlechteren Fehlermeldung.

Teil V – Die Migration

Vorher: Snapshot

```
kubectl -n openbao exec -it openbao-0 -- sh -c \  
  'bao login -method=userpass username=snapshot >/dev/null && \  
    bao operator raft snapshot save /tmp/pre-pkcs11.snap'  
kubectl -n openbao cp openbao-0:/tmp/pre-pkcs11.snap ./pre-pkcs11-2026-09-21.snap
```

Nicht verhandelbar, und mit einer Frist: Ein Snapshot braucht ein **entsiegeltes** OpenBao. Sobald der Pod mit der Seal-Stanza neu startet, ist er sealed, bis die Migration durch ist – das Fenster ist dann zu. Der erste Anlauf hier wurde genau deshalb abgebrochen und der Snapshot nachgeholt.

Deploy und Neustart

```
$ helm upgrade openbao openbao/openbao --version 0.29.4 -n openbao -f helm/values.yaml  
Release "openbao" has been upgraded. Happy Helming!  
REVISION: 8  
  
$ kubectl -n openbao delete pod openbao-0  
$ kubectl -n openbao logs openbao-0 | grep -E 'seal|Seal'  
core: entering seal migration mode; Vault will not automatically unseal even  
      if using an autoseal: from_barrier_type=shamir to_barrier_type=pkcs11  
Auto Seal: pkcs11 (builtin: true, key_label: "openbao-unseal",  
      lib: "/usr/lib/pkcs11/opensc-pkcs11.so", mechanism: "CKM_RSA_PKCS_OAEP",  
      token_label: "SmartCard-HSM")  
  
$ kubectl -n openbao exec openbao-0 -- bao status  
Seal Type           pkcs11  
Recovery Seal Type  shamir  
Initialized          true  
Sealed               true  
Total Recovery Shares 5  
Threshold            3  
Seal Migration in Progress true  
Version              2.6.2+hsm
```

OpenBao hat das HSM gefunden, den Key erkannt, die PIN akzeptiert – und weiß, dass der Root-Key im Storage noch Shamir-verschlüsselt ist. Es wartet.

unseal -migrate

```
kubectl -n openbao exec -it openbao-0 -- bao operator unseal -migrate  
Unseal Key (will be hidden):
```

Dreimal, je ein Shamir-Key. Beim dritten passiert die Arbeit:

```
core: migrating from shamir to auto-unseal: to=pkcs11
core: seal migration complete
core: post-unseal setup complete
```

```
$ bao status
Seal Type          pkcs11
Recovery Seal Type shamir
Sealed             false
Total Recovery Shares 5
Threshold          3
```

Seal Migration in Progress ist weg. Die fünf Keys sind jetzt Recovery Keys. `-migrate` ist damit verbraucht – ein zweites Mal wäre ein Fehler („no migration seal found“, dieselbe Meldung wie bei einem Pod, der die Stanza noch nicht hat; der erste Versuch hier lief gegen genau so einen).

Der Beweis

Migration erfolgreich heißt nicht Auto-Unseal funktioniert. Der Test, um den es geht: Pod löschen, **nichts** eingeben.

```
$ kubectl -n openbao delete pod openbao-0
$ kubectl -n openbao logs openbao-0 | grep -E 'unseal'
core: stored unseal keys supported, attempting fetch
core: vault is unsealed
core: unsealed with stored key

$ kubectl -n openbao get pod openbao-0
NAME          READY   STATUS    RESTARTS   AGE
openbao-0     1/1     Running   0           21s

$ curl -s -o /dev/null -w '%{http_code}\n' https://openbao.example.internal/v1/sys/health
200
```

2,4 Sekunden zwischen `attempting fetch` und `unsealed`. Das ist die RSA-Entschlüsselung im Stick plus vier Schichten dazwischen.

Der Rückweg (nicht durchgeführt)

Sollte das HSM ersetzt werden oder der Seal zurück zu Shamir: Stanza behalten, `disabled = "true"` ergänzen, Pod neu starten, dann `bao operator unseal -migrate` – diesmal mit drei **Recovery** Keys. OpenBao entschlüsselt den Root-Key ein letztes Mal über das HSM und verschlüsselt ihn neu nach Shamir. Voraussetzung: Das HSM ist noch da. Der Rückweg braucht beide Seals gleichzeitig. Das ist der Grund, warum ein sterbendes HSM sofort gehandelt werden muss – nicht, wenn es tot ist.

Dieser Schritt wurde hier nicht ausgeführt. Er gehört in die Restore-Probe aus N° 7, bevor der Seal produktiv gilt.

Teil VI – Was schiefging, und warum

Fehler	Symptom	Ursache	Lösung
Falsches Basis-Image	this build of OpenBao has PKCS#11 disabled	openbao/openbao ist ohne CGO/hsm gebaut	FROM openbao/openbao-hsm:2.6.2
Leeres Secret	pin is required; Secret existiert	zsh: read -p liest vom Koprozess, \$PIN leer	zsh-Syntax read -rs "PIN?...", Länge prüfen
Protokoll-Mismatch	failed to find token; im Debug-Pod „No smart card readers“; Host-Journal Client 4:5, Server 4:4	pcsc-lite ≥ 2.3 im Image vs. pcscd 2.0.3 auf Ubuntu 24.04	pcsc-lite-libs=2.2.3-r1 aus Alpine 3.21 pinnen
Falsches Token-Label	failed to find token with label: SmartCard-HSM (UserPIN)	Label stammt aus OpenSCs PKCS#15-Emulation; 0.26 ≠ 0.23/0.25	Label aus dem Ziel-Image lesen: SmartCard-HSM
Kein Migrations-Seal	can't perform a seal migration, no migration seal found	unseal -migrate gegen einen Pod ohne Stanza (Deploy fehlte)	Reihenfolge: Deploy → Pod → -migrate
Immutable STS	helm upgrade würde volumeClaimTemplates ändern	Kubernetes verbietet das; --dry-run=server prüft es nicht	delete sts --cascade=orphan, dann helm upgrade
Falsches Registry-Präfix (vermieden)	wäre ImagePullBackOff	Chart-Default registry: quay.io	registry: docker.io explizit
Stale Image (vermieden)	wäre altes Image trotz Push	gleicher Tag + IfNotPresent + Node-Cache	Digest in tag

Und zwei, die keine Fehlermeldung hatten: Der vorbereitete Plan sah AES-GCM vor, das der Token nicht kann – aufgefallen erst bei `pkcs11-tool -M`. Und der Token wurde ohne DKEK initialisiert – aufgefallen erst beim Schreiben dieses Leitfadens, bei `sc-hsm-tool`. Beides hätte man vor dem ersten Schlüssel gewusst, wenn man die zwei Kommandos zuerst ausgeführt hätte.

Teil VII – Betrieb und Ausfallverhalten

Was jetzt ausfallen kann

Vor dem Umbau gab es einen Ausfallmodus: Pod neu, jemand tippt Keys. Jetzt gibt es fünf, und keiner ist mit Tippen zu lösen:

Fällt aus	Symptom	Was hilft
worker-03	Pod Pending (nodeSelector), Raft-PVC hängt	Stick in anderen Worker, <code>qm set</code> auf die andere VM, <code>nodeSelector</code> ändern, Polkit-Regel dort
USB-Stick	failed to find token, Health-Check-Warnungen im Log	Ersatzstick – nur mit DKEK-Share (siehe unten)
pcscd auf dem Host	dasselbe Symptom	<code>systemctl restart pcscd.socket</code> ; Journal lesen
Secret openbao-hsm	pin is required	Secret neu anlegen, Pod neu
User-PIN gesperrt	CKR_PIN_LOCKED	<code>sc-hsm-tool --unlock-pin</code> mit SO-PIN

Zur letzten Zeile: Die User-PIN hat **drei Versuche**. Ein Pod im CrashLoop mit falscher PIN im Secret verbraucht sie in unter einer Minute. Danach ist das Token gesperrt, und nur die SO-PIN öffnet es wieder. Wer die PIN ändert, prüft das Secret (Längen-Check!), bevor er den Pod neu startet.

OpenBao führt alle zehn Minuten einen **Seal-Health-Check** aus – verschlüsselt und entschlüsselt einen Zufallswert über das HSM – und warnt im Log, wenn er scheitert. Das ist die Zeile, auf die ein Log-Alarm gehört: Sie kommt Stunden vor dem Neustart, der dann nicht mehr klappt.

Recovery Keys: was sie können und was nicht

- **Können:** einen neuen Root-Token erzeugen (`generate-root`), den Rückweg zu Shamir autorisieren (`unseal -migrate` mit `disabled = "true"`), einen manuell versiegelten OpenBao (`bao operator seal`) wieder öffnen – aber nur, wenn das HSM erreichbar ist; sie autorisieren dann lediglich.
- **Können nicht:** den Root-Key rekonstruieren. Ohne HSM sind die Daten verschlüsselt, und die Recovery Keys sind Papier.

Sie gehören trotzdem in den Passwort-Manager, an denselben Ort wie vorher die Unseal-Keys – und das Runbook aus N° 1 muss umbenennen: Nach einem Neustart ist nichts zu unsealen; wer es versucht, bekommt einen Fehler.

Es sind die alten Keys – und warum man sie jetzt tauschen sollte

Ein Missverständnis, das ich selbst hatte: Bei einer *Neu-Initialisierung* mit HSM-Seal (bao operator init gegen eine frische Instanz) erzeugt OpenBao neue Recovery Keys und gibt sie einmalig aus. Bei einer *Migration* passiert das nicht. unseal -migrate widmet die fünf Shamir-Keys um – dasselbe Schlüsselmaterial, neue Rolle. bao status zeigt es: Recovery Seal Type shamir, Total Recovery Shares 5, Threshold 3 – die alten Parameter.

Das heißt auch: Die Keys, die während der Migration dreimal in ein Terminal getippt wurden, sind die Keys, die künftig den Rückweg autorisieren. Wer sie in dieser Sitzung auf mehreren Rechnern, in einer Bildschirmfreigabe oder in einer Shell-History hatte, tauscht sie jetzt – die Migration ist der natürliche Zeitpunkt, weil man die alten gerade ohnehin zur Hand hat:

```
kubectl -n openbao exec -it openbao-0 -- bao operator rekey \
  -target=recovery -init -key-shares=5 -key-threshold=3
# Ausgabe: Nonce. Dann dreimal, je ein ALTER Recovery Key:
kubectl -n openbao exec -it openbao-0 -- bao operator rekey -target=recovery -nonce=<nonce
>
```

Beim dritten Key gibt OpenBao **fünf neue Recovery Keys** aus – einmalig, im Terminal. Die alten sind ab diesem Moment ungültig. Die neuen gehen in den Passwort-Manager, bevor irgendetwas anderes passiert (N° 7, der Fehler mit den Shell-Variablen). -target=recovery ist der Unterschied zum Rekey eines Shamir-Seals; ohne das Flag versucht OpenBao, die Barrier-Keys zu wechseln, und die gibt es bei einem Auto-Seal nicht.

Ich habe diesen Schritt hier **nicht** ausgeführt – die Keys dieser Instanz haben nur ein Terminal gesehen. Er steht als Verfahren hier, weil er in jedes Runbook gehört, das eine Migration beschreibt.

Der Schlüssel ist nicht gesichert

Das ist der Punkt, den dieser Leitfaden nicht wegerklären kann. Der Token hat keinen DKEK. sc-hsm-tool --wrap-key ist damit nicht möglich; der Seal-Key openbao-unseal existiert genau einmal. Fällt der Stick aus, gibt es keinen Ersatz – und damit keinen Weg an die Raft-Daten, auch nicht über den Snapshot, denn der ist mit demselben Root-Key verschlüsselt.

Für den Homelab-Test ist das akzeptabel. Vor einem Produktiveinsatz ist es nicht akzeptabel, und der Weg dahin ist klar:

1. Zurück zu Shamir migrieren (Teil V, Rückweg) – solange der Stick lebt.
2. Token neu initialisieren mit --dkek-shares 1, Share-Datei erzeugen, importieren, Share und Passwort getrennt sichern.
3. Schlüssel neu erzeugen, exportieren (--wrap-key), Export sichern.
4. Erneut nach PKCS#11 migrieren. Zweiten Stick beschaffen, mit demselben Share initialisieren, Schlüssel importieren, Restore-Probe damit.

Das ist ein Nachmittag. Er ist billiger als der Tag, an dem der Stick nicht mehr blinkt.

Snapshots bleiben Pflicht

Auto-Unseal ändert nichts an N° 7. Der Snapshot-CronJob läuft weiter; die Restore-Probe muss künftig ein Detail mehr kennen: Eine Instanz, die den Snapshot bekommt, braucht denselben Seal – dasselbe HSM oder den Rückweg nach Shamir vor dem Snapshot. Ein Snapshot eines pkcs11-versiegelten OpenBao auf eine Shamir-Drill-Instanz zu spielen, wird abgelehnt.

Prüfen, dass es läuft

```
kubectrl -n openbao get pod openbao-0 -o wide          # 1/1, NODE worker-03
kubectrl -n openbao exec openbao-0 -- bao status       # Seal Type pkcs11, Sealed false
kubectrl -n openbao logs openbao-0 | grep -i 'health'  # keine Warnungen
ssh worker-03 'sudo journalctl -u pcsd --since -1h | grep -ci mismatch' # 0
ssh worker-03 'sc-hsm-tool | grep tries'              # User PIN tries left: 3
```

Teil VIII – Debug: Das Netzwerk-HSM antwortet nicht

Der Stick aus Teil II hängt an einem Worker. Ein HSM, das über das Netz angesprochen wird – Securosys Primus, Thales Luna, Nitrokey NetHSM –, hängt an nichts, und genau deshalb geht bei ihm mehr schief: Zwischen bao und dem Schlüssel liegen dann nicht pcscd und ein Socket, sondern eine **Hersteller-Bibliothek**, die TCP spricht, TLS macht, eine eigene Konfigurationsdatei liest und eigene Logs schreibt – und dazwischen ein Kubernetes-Netz mit DNS, Services und NetworkPolicies.

Dieser Teil ist eine Sitzung, keine Theorie. Nitrokey liefert den NetHSM als Container zum Testen, und die zugehörige PKCS#11-Bibliothek gibt es fertig für musl. Damit lässt sich ein Netzwerk-HSM im Cluster aufsetzen und dann Schicht für Schicht kaputtmachen. Die Reihenfolge, in der man die Schichten prüft, ist nicht verhandelbar: **von außen nach innen, von dumm nach schlau**. Wer mit den OpenBao-Logs anfängt, liest eine Fehlermeldung, die vier Schichten tiefer entstanden ist. Sechs Ausfälle sind in diesem Teil, vier davon waren nicht geplant.

Das Labor

Baustein	Version	Rolle
nitrokey/nethsm:testing	S-Keyfender 5.0	Software-NetHSM, REST-API auf 8443, nur amd64
nethsm-pkcs11	v3.0.0, musl	die Hersteller-Bibliothek; liest /etc/nitrokey/p11nethsm.conf
Image softxpert/openbao-nethsm:2.6.2	openbao-hsm + Bibliothek + OpenSC	OpenBao mit PKCS#11 und pkcs11-tool
dev-Cluster	RKE2, 5× amd64, Cilium	das Netz, in dem es passiert

Zwei Dinge gingen schief, bevor die erste Zeile des Verfahrens dran war. Der NetHSM-Container läuft nur auf amd64; auf dem arm64-kind-Cluster des Macs startete er emuliert und starb mit Failure("same data from timer ...") – sein Entropie-Selbsttest bekommt unter Emulation identische Timer-Werte. Also ein echter amd64-Cluster. Und die Bibliothek hat **keine veröffentlichten Prüfsummen**: checksums.txt im Release ist ein 404. Geprüft wurde per file (ELF, Architektur) – für ein Labor tragbar, für Produktion nicht.

Das Dockerfile, analog zu Teil III:

```

FROM openbao/openbao-hsm:2.6.2
ARG TARGETARCH
USER root
COPY libnethsm_pkcs11-x86_64.so /tmp/x86_64.so
COPY libnethsm_pkcs11-aarch64.so /tmp/aarch64.so
RUN set -eu; case "$TARGETARCH" in amd64) a=x86_64;; arm64) a=aarch64;; esac; \
    mkdir -p /usr/lib/nitrokey /etc/nitrokey && cp /tmp/$a.so /usr/lib/nitrokey/libnethsm_
pkcs11.so && rm /tmp/*.so && \
    apk add --no-cache opensc pcsc-lite-libs # pkcs11-tool for the debug steps
USER openbao

```

Der NetHSM als Deployment mit Service nethsm, ein Job, der ihn provisioniert (Operator-User operator, RSA-2048-Key openbao-unseal mit Mechanismus RSA_Decryption_OAEP_SHA256), die Bibliothekskonfiguration als ConfigMap, die Operator-Passphrase als Secret – alle Werte sind Laborwerte und bewusst abgedruckt:

```

# /etc/nitrokey/p11nethsm.conf (ConfigMap p11nethsm)
log_level: Info
slots:
  - label: LabHSM
    operator:
      username: "operator"
    instances:
      - url: "https://nethsm.nethsm-lab.svc:8443/api/v1"
        danger_insecure_cert: true

```

```

# Auszug aus values-nethsm.yaml
server:
  image: {registry: docker.io, repository: softxpert/openbao-nethsm, tag: "2.6.2@sha256:48
c668b6..."}
  volumes: [{name: p11nethsm, configMap: {name: p11nethsm}}]
  volumeMounts: [{name: p11nethsm, mountPath: /etc/nitrokey, readOnly: true}]
  extraSecretEnvironmentVars:
    - {envName: BAO_HSM_PIN, secretName: openbao-hsm, secretKey: BAO_HSM_PIN}
  standalone:
    config: |
      seal "pkcs11" {
        lib      = "/usr/lib/nitrokey/libnethsm_pkcs11.so"
        token_label = "LabHSM"
        key_label  = "openbao-unseal"
        mechanism  = "CKM_RSA_PKCS_OAEP"
      }

```

Alle Manifeste liegen in cluster/nethsm-lab/.

Die Kette

Pod openbao-lab-0

```
bao  seal "pkcs11" { lib, token_label, key_id, mechanism }
    | dlopen
    | libnethsm_pkcs11.so -- liest --> /etc/nitrokey/p11nethsm.conf
    | HTTPS (Basic Auth: operator / BAO_HSM_PIN)
```

NetworkPolicy · CoreDNS · Service nethsm (Endpoints!)

NetHSM :8443 /api/v1/keys/openbao-unseal/decrypt
└─ eigenes Log: jede Anfrage, jeder Statuscode

Schritt 1 – Kommt man überhaupt hin?

Der erste Start des OpenBao-Pods, noch bevor irgendetwas absichtlich kaputt war:

```
$ kubectl -n nethsm-lab logs openbao-lab-0 | grep nethsm_pkcs11
[INFO nethsm_pkcs11::config::logging] Loaded config file at: /etc/nitrokey/p11nethsm.conf
[INFO nethsm_pkcs11::config::initialization] Loaded configuration with 1 slots
[WARN nethsm_pkcs11::backend::login] Connection attempt 1 failed: IO error connecting to
the instance, io: Connection refused, retrying in 0s
[ERROR nethsm_pkcs11::backend::login] Retry count exceeded after 0 attempts, instance is u
nreachable
[ERROR nethsm_pkcs11::api::token] Error getting info: Api(InstanceRemoved)
Error configuring seal "pkcs11": failed to find token with label: LabHSM
```

Connection refused – Sekunden nachdem der Provisionierungs-Job aus demselben Namespace den NetHSM erreicht hatte. Aus dem Pod, nicht von der Node, mit dem Debug-Pod aus Teil IV (gleiches Image, gleiche UID, gleicher Mount):

```
$ kubectl -n nethsm-lab exec pkcs11-debug -- nslookup nethsm.nethsm-lab.svc
** server can't find nethsm.nethsm-lab.svc: NXDOMAIN
$ kubectl -n nethsm-lab exec pkcs11-debug -- wget -q -O- --no-check-certificate https://ne
thsm:8443/api/v1/health/state
wget: can't connect to remote host (10.43.87.221): Connection refused
```

Zwei Lektionen in zwei Zeilen. Erstens: nslookup in BusyBox fragt den Namen absolut ab und kennt die Search-Domains nicht – nethsm.nethsm-lab.svc ist erst mit .cluster.local ein Name. wget hat ihn über resolv.conf aufgelöst. Zweitens: Der Name löst auf die Service-IP auf, und die lehnt ab. Ein Service lehnt ab, wenn er **keine Endpoints** hat:

```
$ kubectl -n nethsm-lab get pods,endpoints
NAME                READY  STATUS
pod/nethsm-5844c8b95f-xzw75  0/1    Running
NAME                ENDPOINTS
endpoints/nethsm    <none>

$ kubectl -n nethsm-lab logs deploy/nethsm | tail -2
[http.access] request GET /api/v1/health/alive
[http.access] response 412 response time 4.106ms
```

Die Readiness-Probe zeigte auf `/api/v1/health/alive` – und der NetHSM antwortet dort mit 200 nur, solange er *Unprovisioned* oder *Locked* ist. Sobald er *Operational* ist, kommt 412; `/health/ready` verhält sich genau umgekehrt. Der Job kam durch, weil der Pod da noch „ready“ war; die Provisionierung selbst hat ihm die Endpoints entzogen. Ein refused, das mit Firewalls nichts zu tun hat. Lösung: eine TCP-Probe, die in jedem Zustand gilt.

Und noch etwas, das erst beim Nachprüfen auffiel: `wget` gegen die **Pod-IP** statt den Namen scheitert im TLS-Handshake (`tlsv1 alert decode error`) – der NetHSM will SNI. Schritt 1 heißt also: über den Namen testen, den auch die Bibliothek benutzt.

Schritt 2 – Liegt die Bibliothek da, und lädt sie?

```
kubectl -n nethsm-lab exec openbao-lab-0 -- ldd /usr/lib/nitrokey/libnethsm_pkcs11.so
kubectl -n nethsm-lab exec openbao-lab-0 -- ls -la /etc/nitrokey/
```

`ldd` muss jede Zeile auflösen; ein `not found` meldet OpenBao als `CKR_GENERAL_ERROR`, ohne die Abhängigkeit zu nennen. Und die Bibliothek braucht **ihre** Konfiguration. Mit entferntem `ConfigMap-Mount`:

```
[ERROR nethsm_pkcs11::api] NetHSM PKCS#11: Failed to initialize configuration: Failed to load config
Error configuring seal "pkcs11": failed to initialize PKCS11: pkcs11: 0x6: CKR_FUNCTION_FAILED

$ kubectl -n nethsm-lab describe pod openbao-lab-0 | sed -n '/Mounts:/,/Conditions:/p' | grep nitrokey
(nichts)
```

`CKR_FUNCTION_FAILED` ist die unspezifischste Meldung, die PKCS#11 hat. Die Zeile darüber, aus der Bibliothek, ist die, die zählt – und `describe pod` zeigt, was wirklich gemountet ist, nicht, was in den Values steht.

Schritt 3 – Sieht die Bibliothek einen Slot?

```
$ pkcs11-tool --module /usr/lib/nitrokey/libnethsm_pkcs11.so -L
Slot 0 (0x0): NetHSM
  token label      : LabHSM
  token flags      : login required, rng, token initialized, PIN initialized
```

Und mit `danger_insecure_cert: false` gegen das selbstsignierte Zertifikat des Test-Containers:

```
[WARN nethsm_pkcs11::backend::login] Connection attempt 1 failed: IO error connecting to the instance, io: invalid peer certificate: UnknownIssuer
```

Das ist der Fehler, den man in Produktion **haben will** – und dann mit dem CA-Zertifikat der Appliance in der Bibliotheksconfiguration behebt, nicht mit dem `danger_`-Schalter.

Schritt 4 – Kann das Token, was OpenBao braucht?

```
$ pkcs11-tool --module ... -M | grep -E 'AES-GCM|OAEP'  
RSA-PKCS-OAEP, keySize={1024,8192}, hw, sign
```

Kein AES-GCM, wie beim Stick. RSA-OAEP ist da; mechanism = "CKM_RSA_PKCS_OAEP".

Schritt 5 – Login und Schlüssel

Die erste Initialisierung von OpenBao scheiterte – nicht am Netz, nicht an der PIN:

```
$ bao operator init -recovery-shares=5 -recovery-threshold=3  
Error initializing: ... failed to store keys: failed to encrypt keys for storage: no key found
```

```
$ pkcs11-tool --module ... --login -0  
Public Key Object; RSA 2048 bits  
  label:  
  ID:      6f70656e62616f2d756e7365616c  
Private Key Object; RSA  
  label:  
  ID:      6f70656e62616f2d756e7365616c  
Usage:      decrypt, sign  
Access:      sensitive, always sensitive, never extractable
```

Das Label ist **leer**. nethsm-pkcs11 bildet die NetHSM-Key-ID auf CKA_ID ab – als Hex des Strings: 6f70656e62616f2d756e7365616c ist openbao-unseal. key_label = "openbao-unseal" kann nie matchen. Die Stanza braucht key_id = "0x6f70656e62616f2d756e7365616c". Dieselbe Lektion wie in Teil III, andere Bibliothek: Was ein Objekt heißt, entscheidet die Bibliothek, nicht die Doku.

Der Fehlschlag hatte eine Nebenwirkung, die in keinem Runbook steht: OpenBao war danach **halb initialisiert** – Initialized true, Sealed true, Total Recovery Shares 0. Die Barrier war angelegt, die Keys nie gespeichert. Aus diesem Zustand gibt es keinen init mehr und keinen unseal; der Raft-Store musste weg (PVC löschen, Chart neu installieren). In Produktion hieße das: init gegen ein HSM erst, wenn Schritt 1 bis 6 grün sind.

Dann die falsche PIN – im Secret WrongPassphrase:

```
[ERROR nethsm_pkcs11::backend::login] Login check failed: Api(ResponseError(ResponseContent { status: 401, content: "" })))  
[ERROR nethsm_pkcs11::backend] Username not configured for this user  
[WARN nethsm_pkcs11::api::token] C_Login failed with error UserTypeInvalid  
Error configuring seal "pkcs11": failed to login: pkcs11: 0x103: CKR_USER_TYPE_INVALID
```

OpenBao sagt CKR_USER_TYPE_INVALID. Nicht CKR_PIN_INCORRECT. Wer nur den Rückgabewert liest, sucht einen falschen Benutzertyp. Die Wahrheit steht zwei Zeilen höher (401) und im Log des HSM:

```
[http.access] request GET /api/v1/users/operator  
[http.access] response 401 response time 3.632ms
```

Schritt 6 – Eine Operation

```
$ pkcs11-tool --module ... --login --encrypt --id 6f70... -m RSA-PKCS-OAEP --hash-algorithm SHA256 -i /tmp/plain -o /tmp/enc
Using encrypt algorithm RSA-PKCS-OAEP
$ wc -c /tmp/enc
0
```

Null Bytes: nethsm-pkcs11 implementiert C_Encrypt nicht. Das ist kein Fehler des HSM – OpenBao verschlüsselt ohnehin in Software mit dem öffentlichen Schlüssel und lässt nur *entschlüsseln*. Die Signatur als Ersatz:

```
$ pkcs11-tool --module ... --login --sign --id 6f70... -m SHA256-RSA-PKCS -i /tmp/d -o /tmp/sig
[ERROR nethsm_pkcs11::backend] The mechanism RsaPkcs(Some(Sha256)) not supported for PrivateKey openbao-unseal
error: PKCS11 function C_SignInit failed: rv = CKR_MECHANISM_INVALID (0x70)

$ curl -k -u operator:... https://nethsm:8443/api/v1/keys/openbao-unseal
{"mechanisms":["RSA_Decryption_OAEP_SHA256"],"type":"RSA","operations":5,...}
```

Die **Nutzungsmaske**: Der Schlüssel darf genau eins, OAEP-Entschlüsselung. CKR_MECHANISM_INVALID beim Signieren ist also richtig, nicht kaputt – und die API zählt sogar mit, wie oft er benutzt wurde. Der Beweis, dass die Operation geht, ist der Unseal selbst (Schritt 7).

Schritt 7 – Was das HSM selbst sagt

Nach dem Fix aus Schritt 5, init erneut, dann Pod gelöscht:

```
$ kubectl -n nethsm-lab logs openbao-lab-0 | grep unseal
core: stored unseal keys supported, attempting fetch
core: vault is unsealed
core: unsealed with stored key

$ kubectl -n nethsm-lab logs deploy/nethsm | grep -v health | tail -2
[http.access] request POST /api/v1/keys/openbao-unseal/decrypt
[http.access] response 200 response time 5.606ms
```

235 Millisekunden von attempting fetch bis unsealed, und auf der anderen Seite eine Zeile, die sagt, welcher Schlüssel wofür benutzt wurde. Kein Client-Log ist so eindeutig. Bei Securosys ist das das Audit-Log der Partition, bei Luna das Syslog des Geräts – der Zeitstempel des letzten Pod-Starts ist der Suchbegriff.

Schritt 8 – Die Stanza gegen das Gefundene halten

Jetzt erst, und die **gerenderte**:

```
kubectl -n nethsm-lab exec openbao-lab-0 -- cat /openbao/config/extraconfig-from-values.hcl
kubectl -n nethsm-lab exec openbao-lab-0 -- sh -c 'env | grep -E "^BAO_HSM_" | sed "s/=.*/=set>/'
```

lib = was ldd auflöst, token_label = Schritt 3, key_id = Schritt 5, mechanism = Schritt 4, BAO_HSM_PIN gesetzt und kein pin mit anderem Wert in der Stanza – die Umgebungsvariable gewinnt.

Schritt 9 – Mounts und Secrets

```
kubectl -n nethsm-lab describe pod openbao-lab-0 | sed -n '/Mounts:/,/Conditions:/p'
kubectl -n nethsm-lab get secret openbao-hsm -o json | python3 -c "...len..." # Teil IV
```

Was describe zeigt, ist die Wahrheit; was in den Values steht, ist die Absicht (Schritt 2 hat den Unterschied gezeigt).

Schritt 10 – Jetzt die OpenBao-Logs

```
kubectl -n nethsm-lab logs openbao-lab-0 --previous | grep -iE 'seal|pkcs|error'
```

Mit 1–9 im Kopf sind sie lesbar: failed to find token war Schicht 1 (Endpoints) oder 2 (Konfiguration), CKR_USER_TYPE_INVALID war die PIN, no key found war das Label, CKR_FUNCTION_FAILED die fehlende Datei.

Schritt 11 – Von der Node ja, aus dem Pod nein

Eine default-deny-Egress-Policy auf die OpenBao-Pods, drei Zustände, drei verschiedene Symptome:

Policy erlaubt	Bibliothek meldet
nichts	I/O error ... failed to lookup address information: Try again
nur DNS (53 → kube-system)	I/O error ... timeout: global
DNS + 8443 → app: nethsm	core: vault is unsealed

Das erste Symptom sieht aus wie ein DNS-Problem und ist keins: Eine Egress-Policy ohne DNS-Regel tötet die Namensauflösung zuerst. Und der Debug-Pod – anderes Label, nicht von der Policy erfasst – erreichte den NetHSM die ganze Zeit. „Aus dem einen Pod ja, aus dem anderen nein“ ist fast immer eine Policy.

```
kubectl -n nethsm-lab get networkpolicy
kubectl -n nethsm-lab describe networkpolicy default-deny-egress
```

Schritt 12 – strace, und was es diesmal nicht zeigt

Der Plan war: `kubectl debug` mit einem Alpine-Container, `apk add strace`, an `bao` hängen. Drei Dinge lernte ich dabei über Ephemeral-Container:

1. Sie **erben die NetworkPolicy** des Pods. `apk add` hing, weil nur DNS und 8443 erlaubt waren. Ein Image, das `strace` schon hat (`nicolaka/netshoot`), löst das – das zieht die Node, nicht der Pod.
2. Sie laufen mit der `securityContext` des Pods – UID 100, keine Capabilities. `ptrace` braucht Root und `SYS_PTRACE`: `--custom debug-profile.json` mit `runAsUser: 0` und `capabilities.add: [SYS_PTRACE]`.
3. `bao` ist nicht PID 1. PID 1 ist die Shell des Charts, PID 12 `dumb-init`, und PID 13 heißt `ld-musl-x86_64`. – das HSM-Image startet das `glibc`-gebaute Binary über `ld-linux-x86-64.so.2 --preload libccompat.so.0.pgrep -f 'argv0 bao'` findet es.

```
$ strace -f -e trace=openat,connect -p 13
strace: Process 13 attached with 12 threads
(nichts)
```

Und dann nichts. Die Bibliothek hält die HTTPS-Verbindung offen (`max_idle_connection`) – ein `bao operator seal` und der `Unseal` danach erzeugen kein neues `connect()`, und die Konfigurationsdatei wurde beim Start gelesen. `strace` beantwortet die Fragen „welche Datei, welcher Port“ nur, wenn man es **vor** dem Start anhängt – bei einem Pod praktisch nur über ein Wrapper-Skript im Image. Was im geteilten Netz-Namespace immer geht:

```
$ ss -tnp | grep 8443
ESTAB 0 0 10.42.4.134:37846 10.43.87.221:8443 users:(("ld-musl-x86_64.",pid=13,fd=8))
```

Der Prozess, der Socket, die Gegenstelle. Für die meisten Fragen aus Schritt 1 reicht das.

Nebenbei bewiesen: Recovery Keys öffnen nach manuellem Seal

Das Labor wurde mit `bao operator init` **gegen ein HSM** initialisiert – der Weg, den Teil V nicht gegangen ist:

```
$ bao operator init -recovery-shares=5 -recovery-threshold=3 -format=json
{"unseal_keys_b64": [],
 "recovery_keys_b64": [ ... 5 ... ],
```

Keine `Unseal`-Keys, fünf `Recovery Keys`, und `OpenBao` war ohne weiteres `Zutun Sealed false`. Und die Behauptung aus Teil VII, ausprobiert:

```
$ BAO_TOKEN=... bao operator seal
Success! Vault is sealed.
$ bao operator unseal <recovery key 1>    Unseal Progress 1/3
$ bao operator unseal <recovery key 2>    Unseal Progress 2/3
$ bao operator unseal <recovery key 3>    Sealed false
```

Dreimal POST /api/v1/keys/openbao-unseal/decrypt im NetHSM-Log. Die Keys haben autorisiert, das HSM hat entschlüsselt – ohne HSM wären die drei Eingaben ins Leere gegangen.

Die Reihenfolge, als Tabelle

#	Schicht	Kommando	Hier gefunden
1	Netz	nslookup, wget/nc aus dem Pod, get endpoints	Service ohne Endpoints (Probe auf 412); SNI
2	Bibliothek	ldd, file, describe pod	ConfigMap-Mount fehlt → CKR_FUNCTION_FAILED
3	Slot	--list-slots	invalid peer certificate: UnknownIssuer
4	Mechanismus	--list-mechanisms	kein AES-GCM; OAEP da
5	Login/Key	--login -0	leeres Label, ID = Hex → key_id; 401 als CKR_USER_TYPE_INVALID
6	Operation	--sign, HSM-API	Nutzungsmaske: nur OAEP-Decrypt
7	HSM-Log	Appliance	POST .../decrypt 200
8	Stanza	gerenderte HCL + Env	key_label → key_id
9	Pod	describe pod, Secret-Länge	–
10	OpenBao	logs --previous	jetzt lesbar
11	Policy	networkpolicy	drei Symptome, eine Ursache
12	Syscalls	strace, ss -tnp	warme Verbindung: ss statt strace

Und die Regel, die alle zwölf zusammenhält: **Nach jedem gefundenen Fehler wieder bei Schritt 1 anfangen.** Vier der sechs Ausfälle in diesem Teil hatten dieselbe Fehlermeldung in OpenBao (failed to find token) und vier verschiedene Ursachen.

Was das Labor nicht beweist

Ein Test-Container ist kein Gerät. Nicht geprüft: TLS mit echtem CA-Zertifikat statt danger_insecure_cert; Fehlversuchszähler und Sperren – der NetHSM hat keinen sichtbaren, echte Appliances sperren Benutzer nach wenigen Versuchen; Hochverfügbarkeit mit mehreren instances in der Bibliothekskonfiguration; und die Logs von Primus oder Luna, die anders aussehen, aber dasselbe sagen.

Teil IX – Ein HSM für alle Pods: die HSM-VM (Entwurf)

Dieser Teil ist nicht umgesetzt. Er beschreibt, was der nächste Schritt wäre, und wägt die Wege ab.

Das Problem mit dem Aufbau aus Teil IV: Der Stick hängt an einem Worker, also hängt OpenBao an einem Worker. Ein HA-OpenBao mit drei Replicas ist so nicht möglich – drei Pods auf drei Nodes bräuchten drei Sticks mit demselben Schlüssel (DKEK!) oder ein HSM, das über das Netz erreichbar ist. Das Ziel: Eine eigene VM hält den Stick und stellt ihn allen OpenBao-Pods zur Verfügung. Vier Wege, vom einfachsten zum richtigen:

Weg A – Transit-Seal über den VM-OpenBao (empfohlen)

Das war der ursprüngliche Plan für N° 6, und er ist nach diesem Leitfaden attraktiver als vorher. Der OpenBao auf der VM aus N° 3 bekommt den Stick und seal "pkcs11" – genau wie hier, nur als systemd-Dienst statt Pod, ohne Protokoll-Mismatch (Client und Daemon aus demselben Ubuntu), ohne nodeSelector, ohne hostPath. Der Cluster-OpenBao bekommt dann keinen PKCS#11-Seal, sondern einen **Transit-Seal**: Er lässt seinen Root-Key vom VM-OpenBao über dessen Transit-Engine verschlüsseln.

```
Cluster-Pods (beliebig viele, beliebige Nodes)
  seal "transit" { address = "https://bao-vm:8200", key_name = "cluster-unseal", ... }
    | HTTPS, Token mit Policy nur auf transit/*/cluster-unseal
    ▼
VM-OpenBao — seal "pkcs11" — pcsd — Nitrokey HSM 2
```

Vorteile: kein PKCS#11 im Cluster, kein eigenes Image, HA mit drei Replicas sofort möglich, die vorbereiteten Dateien cluster/transit-setup.sh und cluster/values-transit.yaml passen.
Nachteil: eine Kette – der Cluster hängt an der VM, die VM am Stick. Fällt die VM, startet kein Cluster-Pod mehr neu; **läuft** er, läuft er weiter. Der Transit-Token braucht Rotation (periodic token, N° 3 hat das Muster).

Weg B – p11-kit remote: die PKCS#11-Bibliothek übers Netz

p11-kit kann ein PKCS#11-Modul über einen Unix-Socket exportieren:

```
# auf der HSM-VM
p11-kit server --provider /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
  "pkcs11:manufacturer=www.CardContact.de" -n /run/p11-kit/hsm.sock
```

Im Pod lädt OpenBao statt opensc-pkcs11.so die Datei p11-kit-client.so und bekommt P11_KIT_SERVER_ADDRESS=unix:path=/run/p11-kit/hsm.sock – wobei der Socket im Pod von einem Sidecar kommt, das ihn zur VM tunnelt (ssh -L, socat mit TLS, oder ein WireGuard-Netz). p11-kit selbst hat **keine Authentifizierung und keine Verschlüsselung**; die PIN wandert durch diesen Kanal. Der Tunnel ist Pflicht, nicht Option.

Vorteil: OpenBao behält `seal "pkcs11"`, die VM ist ein dummer PKCS#11-Server ohne eigenen Zustand. Nachteil: ein Sidecar pro Pod, ein Tunnel, der vor OpenBao stehen muss, und p11-kit im Image – mit derselben pcsc-lite-Frage auf der VM-Seite. Nicht getestet.

Weg C – USB/IP

Der Linux-Kernel kann USB-Geräte über IP exportieren (`usbip`). Die HSM-VM exportiert den Stick, ein Worker importiert ihn und sieht ihn als lokales Gerät. Das verschiebt den Stick von einem Worker zum anderen, macht ihn aber nicht für **alle** Pods gleichzeitig verfügbar – ein USB-Gerät hat einen Host. Für Failover brauchbar, für HA nicht. Und USB/IP über ein unsicheres Netz ist ein unverschlüsselter USB-Bus über ein unsicheres Netz.

Weg D – Nitrokey NetHSM

Nitrokeys Antwort auf genau diese Frage: ein HSM mit REST-API, eigener PKCS#11-Bibliothek (`libnethsm_pkcs11.so`, für `glibc` und `musl`), Clustering, Backup und einer offiziellen OpenBao-Anleitung. Auch dort gilt RSA-OAEP (kein AES-GCM), auch dort ein eigenes Image auf `openbao-hsm`-Basis. Es gibt ein Container-Test-Image für die Probe. Der Preis ist der einer Appliance; für ein Homelab die Antwort auf „wie machen das die Großen“, nicht auf „was baue ich nächste Woche“.

Empfehlung

Weg A. Er nutzt, was da ist (VM-OpenBao, Ansible-Rolle, vorbereitete Skripte), er bringt HA für den Cluster, und er hält PKCS#11 an einem Ort, an dem es einfach ist. Der Stick zieht von `worker-03` auf die VM um (`qm set`), der Cluster-OpenBao migriert von `pkcs11` nach `transit` – eine Auto-Seal-zu-Auto-Seal-Migration, für die OpenBao `disabled = "true"` an der alten Stanza vorsieht. Das wird N° 6b oder ein Nachtrag zu diesem Leitfaden.

Teil X – OpenBao 2.7: vom HSM-Build zum KMS-Plugin (Entwurf)

Auch dieser Teil ist nicht umgesetzt; 2.7.0 ist zum Zeitpunkt des Schreibens Beta.

Der erste Start des HSM-Images hatte eine Warnung im Log, die man lesen sollte:

```
[WARN] The HSM distribution of OpenBao is discontinued and will no longer
receive updates beyond this minor version. PKCS#11 support has not been
removed, but is now available via an external KMS plugin that is drop-in
compatible with the previously built-in PKCS#11 seal.
```

openbao/openbao-hsm endet mit 2.6.x. Ab 2.7 gibt es ein Standard-Image und einen **KMS-Plugin-Mechanismus**: Auto-Seals, die nicht mehr eingebaut sind, laufen als eigener Prozess, den OpenBao startet. Die Konfiguration bekommt eine Stanza dazu, die Seal-Stanza bleibt:

```
plugin "kms" "pkcs11" {
  command = "openbao-plugin-kms-pkcs11"
  version = "v0.2.0"
}

seal "pkcs11" {
  lib      = "/usr/lib/pkcs11/opensc-pkcs11.so"
  token_label = "SmartCard-HSM"
  key_label  = "openbao-unseal"
  mechanism  = "CKM_RSA_PKCS_OAEP"
}
```

Der Name in `seal` muss der Name aus `plugin` sein. kms-Plugins lassen sich **nur** deklarativ registrieren, nicht über die API – sie müssen da sein, bevor OpenBao entsiegelt ist. Das Binary liegt in `plugin_directory`; alternativ zieht OpenBao es als OCI-Artefakt (`image = "...@sha256:..."`).

Die Stolpersteine, die man jetzt schon sieht

Das Plugin-Release (openbao-plugins, kms-pkcs11-v0.2.0, September 2026) liefert openbao-plugin-kms-pkcs11_linux_amd64_v1 – ein **glibc**-Binary (interpreter `/lib64/ld-linux-x86-64.so.2`). Auf dem Alpine-Standard-Image (musl) läuft es nicht ohne `gcompat`. Der saubere Weg ist die UBI-Variante:

```
FROM openbao/openbao-ubi:2.7.x
USER root
RUN microdnf install -y opensc pcsc-lite-libs && microdnf clean all
COPY --chmod=0755 openbao-plugin-kms-pkcs11_linux_amd64_v1 /openbao/plugins/openbao-plugin-kms-pkcs11
USER openbao
```

Zwei Dinge daran wären zu prüfen, sobald 2.7.0 da ist:

- **RHEL 9 liefert pcsc-lite 1.9.x** – Protokoll 4:4. Der Pin aus Teil III würde damit entfallen; der Host-pcscd 2.0.3 passt.
- **OpenSC 0.23 in RHEL 9** – das Token-Label wird wieder SmartCard-HSM (UserPIN) heißen. Nach dem Rebuild prüfen, wie immer.
- `plugin_directory` in der Server-Konfiguration muss auf das Verzeichnis zeigen; der Chart hat dafür `server.standalone.config` und – wenn nötig – ein `extraVolumes/volumes`-Mount für die Plugin-Binaries.

Die Migration selbst soll ohne `unseal --migrate` auskommen: Das Plugin ist laut Release Notes „drop-in compatible“, der Root-Key bleibt mit demselben HSM-Schlüssel gewrappt. Testen würde man das trotzdem im `kind`-Kontext zuerst – mit einem Snapshot davor.

Anhang A – Alle Kommandos

```
# — Proxmox-Host —————
lsusb | grep -i nitrokey
qm set <vmid> -usb0 host=20a0:4230
qm shutdown <vmid> && qm start <vmid>

# — Worker-VM —————
sudo apt install -y pcscd libccid pcsc-tools opensc
sudo systemctl enable --now pcscd
sudo vi /etc/polkit-1/rules.d/40-allow-pcscd.rules          # siehe Teil II
sudo systemctl restart polkit pcscd
pcsc_scan
sc-hsm-tool                                                # Status, DKEK, PIN-Versuche
sc-hsm-tool --initialize --dkek-shares 1 --label openbao  # NUR bei frischem Stick
sc-hsm-tool --create-dkek-share dkek-share-1.pbe
sc-hsm-tool --import-dkek-share dkek-share-1.pbe
pkcs11-tool -M | grep -E 'AES|OAEP'                      # was das Token kann
pkcs11-tool --login --keypairgen --key-type rsa:2048 --label openbao-unseal --id 10 --usage-decrypt
pkcs11-tool -0                                             # öffentlicher Teil
sudo journalctl -u pcscd --since -15min                  # Protokoll-Mismatch?

# — Image —————
docker buildx build --platform linux/amd64,linux/arm64 -t softxpert/openbao-pkcs11:2.6.2 --push .
docker buildx imagetools inspect softxpert/openbao-pkcs11:2.6.2 # Digest → values
trivy image --platform linux/amd64 --severity HIGH,CRITICAL softxpert/openbao-pkcs11:2.6.2

# — Cluster: Vorbereitung —————
kubectl apply -f pkcs11-debug.yaml                        # Teil IV
kubectl -n openbao exec pkcs11-debug -- pkcs11-tool -L    # Label aus DEM Image
read -rs "PIN?HSM User-PIN: "; printf '%s' "$PIN" > /tmp/hsm-pin; unset PIN
kubectl -n openbao create secret generic openbao-hsm --from-file=BA0_HSM_PIN=/tmp/hsm-pin
rm -P /tmp/hsm-pin
kubectl -n openbao get secret openbao-hsm -o json | python3 -c "...len..." # 6-15 Bytes!

# — Cluster: Migration —————
kubectl -n openbao exec -it openbao-0 -- bao operator raft snapshot save /tmp/pre-pkcs11.snap
kubectl -n openbao cp openbao-0:/tmp/pre-pkcs11.snap ./pre-pkcs11.snap
helm upgrade openbao openbao/openbao --version 0.29.4 -n openbao -f helm/values.yaml
kubectl -n openbao delete pod openbao-0
kubectl -n openbao logs openbao-0 | grep -iE 'seal|error'
kubectl -n openbao exec -it openbao-0 -- bao operator unseal -migrate # x3, EINMAL
kubectl -n openbao exec openbao-0 -- bao status

# — Der Beweis —————
kubectl -n openbao delete pod openbao-0                  # nichts eingeben
kubectl -n openbao logs openbao-0 | grep unseal          # "unsealed with stored key"

# — Recovery Keys tauschen (nicht durchgeführt) —————
kubectl -n openbao exec -it openbao-0 -- bao operator rekey -target=recovery -init -key-shares=5 -key-threshold=3
kubectl -n openbao exec -it openbao-0 -- bao operator rekey -target=recovery -nonce=<nonce>
> # x3 ALTE Keys → 5 NEUE

# — Rückweg (nicht durchgeführt) —————
```

```
#   seal "pkcs11" { ... disabled = "true" } → helm upgrade → delete pod  
kubectl -n openbao exec -it openbao-0 -- bao operator unseal -migrate # x3 RECOVERY Keys
```

Anhang B – Glossar

Seal / Barrier – Die Barrier ist OpenBaos Verschlüsselungsschicht über dem Storage; der Seal bestimmt, wie der Root-Key dafür geschützt ist: Shamir (Keys eingeben) oder Auto-Seal (externes System fragen).

Root-Key – Schlüssel, mit dem OpenBao den Storage verschlüsselt. Liegt selbst verschlüsselt im Storage – bei pkcs11 mit einem AES-Schlüssel, der RSA-OAEP-gewrappt ist.

Recovery Keys – Die ehemaligen Unseal-Keys nach einer Migration zu Auto-Seal. Autorisieren generate-root und den Rückweg; können den Root-Key nicht rekonstruieren.

HSM – Hardware Security Module: erzeugt, speichert und nutzt Schlüssel, ohne sie herauszugeben.

Nitrokey HSM 2 / SmartCard-HSM – USB-Stick mit einer SmartCard-HSM- Karte von CardContact; OpenSC-Unterstützung, RSA/ECC, kein AES-GCM.

PKCS#11 (Cryptoki) – Standard-C-API für HSMs und Smartcards; eine .so, die per dlopen geladen wird.

Slot / Token / Objekt – Steckplatz / Karte im Steckplatz / Schlüssel oder Zertifikat auf der Karte. Token haben ein Label; das Label des SmartCard-HSM erzeugt OpenSC und ändert es zwischen Versionen.

Mechanismus – Ein kryptografisches Verfahren, das das Token ausführt (CKM_RSA_PKCS_OAEP, CKM_AES_GCM). pkcs11-tool -M listet sie.

User-PIN / SO-PIN – Login für Schlüsseloperationen (3 Versuche) / Verwaltungs-Login für Initialisierung und PIN-Reset (15 Versuche).

DKEK – Device Key Encryption Key des SmartCard-HSM. Nur mit DKEK-Share lassen sich Schlüssel verschlüsselt exportieren und auf einen zweiten Stick bringen. Wird bei --initialize festgelegt.

OpenSC – Open-Source-Implementierung von PKCS#11 für Smartcards; liefert opensc-pkcs11.so, pkcs11-tool, sc-hsm-tool.

PC/SC, pcscd, libpcsc-lite – Die Reader-Abstraktion unter PKCS#11. pcscd ist der Daemon (spricht CCID/USB), libpcsc-lite die Client- Bibliothek; beide sprechen ein versioniertes Protokoll über /run/pcscd/pcscd.comm. Client 4:5 gegen Server 4:4 wird abgelehnt.

CCID – Chip Card Interface Device, das USB-Protokoll für Smartcard- Reader; libccid ist der Treiber in pcscd.

Polkit – Autorisierungsdienst auf Linux; pcscd fragt ihn, ob ein Prozess den Reader benutzen darf.

HSM-Build (+hsm, cgo) – OpenBao-Binary mit einkompiliertem PKCS#11- Support; als openbao/openbao-hsm bis 2.6.x, danach KMS-Plugin.

KMS-Plugin – Ab OpenBao 2.6/2.7: Auto-Seal-Mechanismus als externer Prozess, deklariert mit plugin "kms" "<name>" in der Server-Konfiguration.

-migrate – Flag von `bao operator unseal`, das den Root-Key vom alten auf den neuen Seal umschlüsselt. Einmal pro Migration, mit den Keys des **alten** Seals.

OnDelete – StatefulSet-Update-Strategie: ein geändertes Template wird erst wirksam, wenn der Pod gelöscht wird. Chart-Default für OpenBao.

Digest-Pin – Image-Referenz `name:tag@sha256:...`; der Digest gewinnt. Schützt vor überschriebenen Tags und Node-Cache.

Über den Autor

Thomas Zachmann ist freiberuflicher Platform Engineer in Hamburg. Er baut Enterprise-Plattformen für Kubernetes, Cloud und AI-Workloads – von Identity und Secrets über CI/CD und GitOps bis Observability – so, dass das interne Team sie danach ohne ihn betreiben kann. Diese Field Notes entstehen aus dieser Arbeit. Für Projektanfragen: thomaszachmann.de.

Der Pod fand das HSM nicht. Der Test-Pod daneben schon.

Dieser Leitfaden schafft das Unseal-Ritual aus N° 1 ab: Der Root-Key liegt danach verschlüsselt mit einem RSA-Schlüssel, der einen Nitrokey HSM 2 nie verlässt. Der Stick hängt am Kubernetes-Worker, der Pod bekommt den pcscd-Socket als hostPath, ein eigenes Image bringt OpenBaos HSM-Build und OpenSC mit. Für Leserinnen und Leser ohne HSM-Erfahrung erklärt Teil I, was Seal, Token, SO-PIN und DKEK bedeuten – und warum die letzte Frage vor dem ersten Schlüssel kommt.

Der Weg dahin: das offizielle Image kann kein PKCS#11; pcsc-lite 2.4 im Container spricht 4:5, pcscd 2.0.3 auf dem Host nur 4:4; OpenSC 0.26 nennt den Token anders als 0.23; ein read -p in zsh liefert ein leeres Secret. Dann unseal -migrate, dreimal – und ein Pod, der ohne Key hochkommt. Zwei Entwürfe zum Schluss: ein HSM für alle Pods, und der Plugin-Weg von OpenBao 2.7.

```
-- Was der Pod sagt, was der Host sagt
Error configuring seal "pkcs11":
  failed to find token with label: SmartCard-HSM (UserPIN)

$ journalctl -u pcscd
Communication protocol mismatch!
Client protocol is 4:5 # pcsc-lite 2.4, Alpine 3.24
Server protocol is 4:4 # pcscd 2.0.3, Ubuntu 24.04
```

01	02	03	04	05	06
Begriffe	Der Stick	Das Image	Protokoll 4:5	Migration	Netzwerk-HSM
Seal, HSM, PKCS#11, DKEK	Proxmox, pcscd, Polkit, RSA-Key	openbao-hsm + OpenSC, multi-arch	pcscd lehnt den Client ab	unseal -migrate x3, der Beweis	NetHSM-Labor, 12 Schritte, 6 Ausfälle

openbao	2.6.2+hsm · chart 0.29.4 · standalone
hsm	Nitrokey HSM 2 · FW 4.1 · RSA-2048 · OAEP
image	hsm-build · opensc 0.26 · pcsc-lite 2.2.3
host	Ubuntu 24.04 · pcscd 2.0.3 · polkit
seal	pkcs11 · recovery shamir 3/5

44 Seiten · 10 Teile · 2 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.
CC BY 4.0 · Weitergabe erlaubt · ohne Gewähr